

FAST — NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES

Department of Artificial Intelligence & Data Science · Karachi Campus

PROGRAMMING FOR ARTIFICIAL INTELLIGENCE

Laboratory Manual

LAB 02 & 03

Program Flow, Comprehensions and Data Structures

Conditionals · Loop Patterns · break & continue · List, Dict and Set Comprehensions
Lists · Tuples · Dictionaries · Sets · Choosing the Right Structure

PREPARED BY

Khadeejah Ashraf

Lab Instructor | Department of Artificial Intelligence & Data Science

Contents

1. Lab Objectives
2. Lab Rules and Conduct
3. Before You Begin — What Lab 01 Assumed

PART A — PROGRAM FLOW AND COMPREHENSIONS

4. Conditional Statements in Depth
5. Loops in Depth
6. `break`, `continue` and `pass`
7. Standard Loop Patterns
8. Comprehensions

PART B — PYTHON'S PRIMARY DATA STRUCTURES

9. Lists
10. Tuples
11. Dictionaries
12. Sets
13. Choosing the Right Structure
14. Quick Reference
15. Class Tasks (to be completed during the session)
16. Home Tasks
17. Submission and Assessment

1. Lab Objectives

This manual covers two lab sessions. Part A is the work of Lab 02 and Part B the work of Lab 03; the class tasks at the end draw on both. By the end of the two sessions you should be able to:

- Write multi-way decisions with `if / elif / else`, nest them sensibly, and replace a short nested decision with a **conditional expression**.
- Choose correctly between a `for` loop and a `while` loop, and iterate with `range()`, `enumerate()` and `zip()`.
- Redirect the flow of a loop with `break`, `continue` and `pass`, and explain exactly where each one jumps to.
- Recognise and write the standard loop patterns: accumulating, counting, searching, tracking a maximum, and sentinel-controlled input.
- Rewrite a simple loop as a **list comprehension**, add a filter to it, and write **dictionary** and **set** comprehensions.
- Judge when a comprehension improves readability and when an ordinary loop is the better choice.
- Use the methods of lists, tuples, dictionaries and sets confidently, and explain the difference between copying a list and merely aliasing it.
- Unpack tuples, return several values from one expression, and use a tuple where an immutable value is required.
- Build, update, nest and iterate dictionaries, and use the frequency-counting pattern.
- Apply set operations — union, intersection, difference and symmetric difference — to remove duplicates and compare collections.
- Select the appropriate data structure for a task, with reference to ordering, mutability, duplicates and lookup cost.

2. Lab Rules and Conduct

- Maintain discipline throughout the lab session.
- Listen carefully and follow the instructions as they are given.
- Raise your hand if you face any problem; do not leave your seat.
- **Class tasks** must be completed and demonstrated during the session itself.
- **Home tasks** must be submitted before the deadline announced on the course portal.
- Get your work checked by the instructor before the end of the session.

Reminder: work that is not checked during the session carries no marks, even if the code runs correctly.

3. Before You Begin — What Lab 01 Assumed

Everything in this manual builds directly on Lab 01. Run the cell below in a notebook; if any line surprises you, revise the relevant section of Lab 01 before going further.

```
marks = [78, 45, 90]           # a list
record = {'Physics': 78, 'Maths': 90} # a dictionary
print(len(marks), sum(marks) / len(marks))
print(marks[0], marks[-1], marks[0:2])
print(record['Maths'], 'Physics' in record)

for m in marks:
    if m >= 50:
        print(m, "pass")
    else:
        print(m, "fail")
```

From Lab 01 you should already know	Used in this manual for
Variables, <code>input()</code> and type conversion	Reading data before it is stored in a structure
Arithmetic, comparison and logical operators	The conditions inside every <code>if</code> , <code>while</code> and comprehension filter
<code>if/elif/else</code>	Multi-way decisions and filters
<code>for</code> , <code>while</code> and <code>range()</code>	Every pattern in Part A
Lists and dictionaries, indexing and slicing	The starting point of Part B

PART A — PROGRAM FLOW AND COMPREHENSIONS

Lab 02 · Sections 4 to 8

4. Conditional Statements in Depth

A decision structure evaluates an expression that is either true or false and chooses which block of statements to run. Lab 01 introduced the syntax; this section is about using it well.

4.1 The `elif` Chain

Conditions in an `if / elif` chain are tested strictly from the top, and the chain stops at the **first** condition that is true. Everything below it is skipped, whether or not it would also have been true. This is why the order of the branches matters.

```
marks = 85

if marks >= 50:
    grade = 'Pass'          # this is true first, so the chain stops here
elif marks >= 80:
    grade = 'Distinction'  # never reached, even though 85 >= 80

print(grade)
```

Output

```
Pass
```

Rule: in a chain of ranges, always test the **narrowest** or highest condition first. Reversing the two branches above fixes the bug.

4.2 Nested Decisions

An `if` may contain another `if`. Nesting is useful when the inner question only makes sense once the outer one has been answered, but each extra level costs the reader an indentation, so keep it shallow.

```
marks = 85
attendance = 60

if attendance >= 75:
    if marks >= 80:
        print("Distinction")
    else:
        print("Pass")
else:
    print("Not eligible – attendance short")
```

When both conditions are independent, a combined condition with `and` says the same thing in one level:

```
if attendance >= 75 and marks >= 80:
    print("Distinction")
```

4.3 The Conditional Expression

Python has a one-line form of `if / else` that produces a **value** rather than performing an action. It is written `A if condition else B` and is the ideal way to choose between two values in a single statement — you will meet it again inside comprehensions in Section 8.

```
n = 7

# the long form
if n % 2 == 0:
    kind = "even"
else:
    kind = "odd"

# the same thing as a conditional expression
kind = "even" if n % 2 == 0 else "odd"
print(n, "is", kind)
```

Output

```
7 is odd
```

Careful: use the conditional expression only when both outcomes are short values. If either branch needs several statements, write the ordinary `if / else`.

4.4 Truthiness in Conditions

A condition does not have to be a comparison. Python treats `0`, `0.0`, the empty string, the empty list, the empty dictionary, the empty set and `None` as false, and everything else as true — which gives a neat way to ask whether a collection has anything in it.

```
items = []

if items:
    print("the list has", len(items), "items")
else:
    print("the list is empty")

name = input_value = ""
print(bool(0), bool(""), bool([]), bool({}), bool(None))
print(bool(-1), bool("0"), bool([0]))
```

Output

```
the list is empty
False False False False False
True True True
```

Note: the string `"0"` and the list `[0]` are **not** empty, so both are true. Only emptiness makes a container false.

5. Loops in Depth

5.1 for versus while

Use a...	When...	Typical header
for loop	you are walking through a known sequence, or repeating a known number of times	<code>for item in collection:</code>
while loop	you repeat until something changes, and cannot say in advance how many passes that will take	<code>while condition:</code>

```
# for: the number of passes is fixed by the sequence
for subject in ['Physics', 'Chemistry', 'Maths']:
    print(subject)

# while: the number of passes depends on the data
total = 0
count = 0
while total < 100:
    total += 15
    count += 1
print("took", count, "passes to pass 100")
```

Output

```
Physics
Chemistry
Maths
took 7 passes to pass 100
```

5.2 enumerate() — the index and the item together

A common beginner habit is to loop over `range(len(items))` just to get the position. `enumerate()` gives the position and the value at the same time, and reads far better.

```
subjects = ['Physics', 'Chemistry', 'Maths']

for i in range(len(subjects)):      # works, but clumsy
    print(i, subjects[i])

for i, subject in enumerate(subjects):    # preferred
    print(i, subject)

for rank, subject in enumerate(subjects, start=1):  # start counting at 1
    print(rank, subject)
```

Output

```
0 Physics
1 Chemistry
2 Maths
0 Physics
1 Chemistry
2 Maths
1 Physics
2 Chemistry
3 Maths
```

5.3 zip() — walking two sequences side by side

`zip()` pairs the items of two or more sequences and stops as soon as the shortest one is exhausted.

```
subjects = ['Physics', 'Chemistry', 'Maths']
marks    = [78, 45, 90]

for subject, mark in zip(subjects, marks):
    print(subject, "->", mark)

print(dict(zip(subjects, marks)))    # a quick way to build a dictionary
```

Output

```
Physics -> 78
Chemistry -> 45
Maths -> 90
{'Physics': 78, 'Chemistry': 45, 'Maths': 90}
```

5.4 The else Clause of a Loop

Both loop forms may carry an `else` block. It runs when the loop finishes **normally** — that is, when the sequence is exhausted or the condition becomes false — and is skipped when the loop is left through a `break`. It is the tidiest way to say “I searched the whole thing and found nothing”.

```
numbers = [4, 9, 16, 25]
target = 7

for n in numbers:
    if n == target:
        print("found", target)
        break
else:
    print(target, "is not in the list")
```

Output

7 is not in the list

6. break, continue and pass

Keyword	Effect
break	Leaves the innermost loop immediately; the loop's else block is skipped.
continue	Abandons the rest of the current pass and jumps back to the loop header for the next item.
pass	Does nothing at all. It is a placeholder that keeps a block syntactically valid while you write the rest.

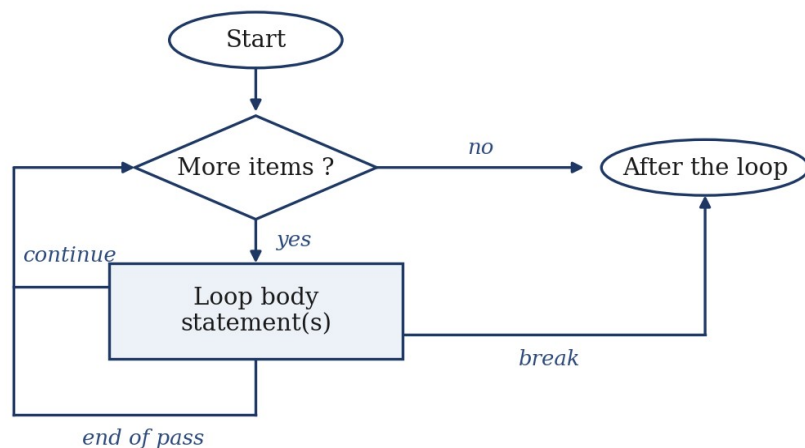


Figure 6.1 Where break and continue send the flow of control

```

for n in range(1, 12):
    if n % 2 == 0:
        continue          # skip the even numbers
    if n > 7:
        break             # stop once past 7
    print(n, end=' ')
print("\ndone")
  
```

Output

```

1 3 5 7
done
  
```

break only leaves **one** level of loop. In a nested loop it returns control to the outer loop, not to the statement after both loops:

```

for i in range(1, 4):
    for j in range(1, 4):
        if j == 2:
            break          # ends the inner loop only
        print(i, j)
  
```

Output

```

1 1
2 1
3 1
  
```

`pass` is not a loop keyword at all, but it belongs beside them because it solves the same practical problem — Python refuses an empty block.

```
for n in range(3):
    pass          # nothing to do yet; the loop is still valid Python

if False:
    pass
```

7. Standard Loop Patterns

Almost every loop you write in this course is one of the five shapes below. Learn them as patterns and most exercises become a matter of choosing the right one.

7.1 Accumulating

Start from a neutral value — 0 for a sum, 1 for a product, "" or [] for a collection — and update it once per pass.

```
marks = [78, 45, 90, 62]

total = 0
for m in marks:
    total += m

print("total  :", total)
print("average:", total / len(marks))
```

Output

```
total  : 275
average: 68.75
```

7.2 Counting

The same shape, but the counter only advances when a condition holds.

```
marks = [78, 45, 90, 62]

passed = 0
for m in marks:
    if m >= 50:
        passed += 1

print(passed, "of", len(marks), "passed")
```

Output

```
3 of 4 passed
```

7.3 Searching

Walk until you find what you are looking for, then stop. The loop `else` reports the failure.

```
subjects = ['Physics', 'Chemistry', 'Maths']
wanted = 'Maths'

for i, subject in enumerate(subjects):
    if subject == wanted:
        print(wanted, "found at index", i)
        break
else:
    print(wanted, "not found")
```

Output

```
Maths found at index 2
```

7.4 Tracking a Maximum or Minimum

Assume the first item is the best so far, then let the loop correct that assumption.

```
marks = {'Physics': 78, 'Chemistry': 45, 'Maths': 90}

best_subject = None
best_mark = -1
for subject, mark in marks.items():
    if mark > best_mark:
        best_subject, best_mark = subject, mark

print("highest:", best_subject, best_mark)
```

Output

```
highest: Maths 90
```

Shortcut: for a plain list, `max(marks)` and `min(marks)` do this for you, and `max(marks, key=marks.get)` finds the dictionary key with the largest value. Write the loop when you also need something else, such as the position.

7.5 Sentinel-Controlled Input

A while loop that keeps reading until the user types an agreed stopping value. This is the standard shape for menu-driven programs, and you will need it in Class Task C1.

```
numbers = []

while True:
    entry = input("Enter a number (0 to finish): ")
    value = int(entry)
    if value == 0:          # the sentinel
        break
    numbers.append(value)

print("you entered", len(numbers), "numbers:", numbers)
```

Output

```
Enter a number (0 to finish): 12
Enter a number (0 to finish): 7
Enter a number (0 to finish): 0
you entered 2 numbers: [12, 7]
```

7.6 Nested Loops and Patterns

The inner loop completes every one of its passes for each single pass of the outer loop, which is what makes it the natural tool for grids and printed patterns.

```
# a right-angled triangle
for i in range(1, 6):
    for j in range(i):
        print("*", end="")
    print()

# the same thing using string repetition
for i in range(1, 6):
    print("*" * i)
```

Output

```
*
**
***
****
*****
```

8. Comprehensions

A comprehension is a single expression that builds a whole collection. It is Python's idiomatic replacement for the very common loop that creates an empty list, walks a sequence, and appends one item per pass.

8.1 From a Loop to a Comprehension

```
numbers = [1, 2, 3, 4, 5, 6]

# the loop you already know
squares = []
for n in numbers:
    squares.append(n ** 2)

# exactly the same result, written as a comprehension
squares = [n ** 2 for n in numbers]

print(squares)
```

Output

```
[1, 4, 9, 16, 25, 36]
```

8.2 The Anatomy of a Comprehension

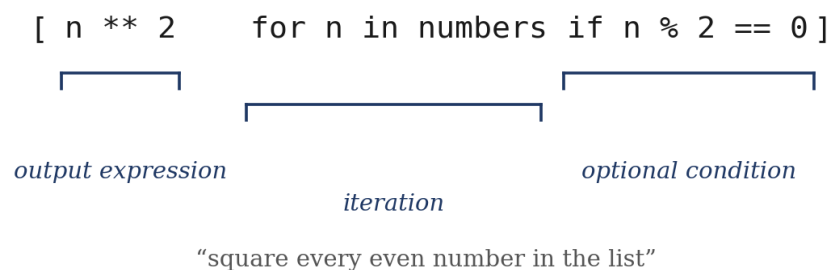


Figure 8.1 The three parts of a list comprehension

Read it from the middle outwards: **for each `n` in `numbers`, if that `n` is even, put the square of `n` into the new list.** The square brackets say the result is a list.

```
numbers = [1, 2, 3, 4, 5, 6]
print([n ** 2 for n in numbers if n % 2 == 0])
```

Output

```
[4, 16, 36]
```

8.3 Filtering with if

The trailing `if` decides which items reach the new list. Items that fail the test are simply not produced — the comprehension is shorter than its source, never longer.

```
words = ['python', 'ai', 'lab', 'comprehension', 'data']

long_words = [w for w in words if len(w) > 3]
```

```
upper_long = [w.upper() for w in words if len(w) > 3]

print(long_words)
print(upper_long)
```

Output

```
['python', 'comprehension', 'data']
['PYTHON', 'COMPREHENSION', 'DATA']
```

8.4 Choosing Between Two Values

A **conditional expression** (Section 4.3) may be used as the output expression. Note the position: `if ... else` in front of the `for` chooses a value, while a bare `if` after the `for` filters.

```
numbers = [1, 2, 3, 4, 5, 6]

labels = ["even" if n % 2 == 0 else "odd" for n in numbers] # chooses
evens = [n for n in numbers if n % 2 == 0]               # filters

print(labels)
print(evens)
```

Output

```
['odd', 'even', 'odd', 'even', 'odd', 'even']
[2, 4, 6]
```

Form	Position	Effect
[A if c else B for x in seq]	Before the for	Every item produces a value; the condition picks which one
[A for x in seq if c]	After the for	Only the items that satisfy the condition appear

8.5 Nested Comprehensions

A comprehension may contain more than one `for`, written in the same order as the nested loops it replaces. It may also produce a list of lists, which is how a small matrix is built.

```
# a multiplication grid: three rows of four columns
grid = [[row * col for col in range(1, 5)] for row in range(1, 4)]
print(grid)

# flattening a list of lists
flat = [value for row in grid for value in row]
print(flat)
```

Output

```
[[1, 2, 3, 4], [2, 4, 6, 8], [3, 6, 9, 12]]
[1, 2, 3, 4, 2, 4, 6, 8, 3, 6, 9, 12]
```

8.6 Dictionary Comprehensions

Curly braces and a key: value output expression build a dictionary instead of a list.

```
words = ['python', 'ai', 'lab']

lengths = {w: len(w) for w in words}
squares = {n: n ** 2 for n in range(1, 6)}
print(lengths)
```

```
print(squares)

# inverting an existing dictionary
marks = {'Physics': 78, 'Chemistry': 45}
print({mark: subject for subject, mark in marks.items()})
```

Output

```
{'python': 6, 'ai': 2, 'lab': 3}
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
{78: 'Physics', 45: 'Chemistry'}
```

8.7 Set Comprehensions

Curly braces with a single output expression build a set, so duplicates disappear on the way in.

```
marks = [78, 45, 90, 45, 78]

print({m for m in marks})          # unique marks
print({m // 10 * 10 for m in marks}) # unique tens band
print({w[0] for w in ['ali', 'ahmed', 'bilal']}) # unique first letters
```

Output

```
{90, 45, 78}
{40, 90, 70}
{'a', 'b'}
```

Note: a set has no order, so the printed order of these results may differ from run to run. Never rely on it.

8.8 When Not to Use a Comprehension

A comprehension should make a short piece of code clearer. Keep the ordinary loop when the body needs several statements, when you are calling `print()` rather than collecting a value, or when the line has grown so long that a reader cannot take it in at a glance.

```
# poor: a comprehension used for its side effect only
[print(n) for n in range(3)]      # builds a useless list of None

# good: an ordinary loop
for n in range(3):
    print(n)
```

PART B — PYTHON'S PRIMARY DATA STRUCTURES

Lab 03 · Sections 9 to 13

9. Lists

A list is an **ordered**, **changeable** sequence that allows duplicates. It is the default choice whenever you simply need to keep several values together.

9.1 The Methods You Will Actually Use

Method	What it does	Example on <code>L = [3, 1, 2]</code>
<code>L.append(x)</code>	Adds one item to the end	<code>L.append(4) → [3, 1, 2, 4]</code>
<code>L.extend(other)</code>	Adds every item of another sequence	<code>L.extend([4, 5]) → [3, 1, 2, 4, 5]</code>

Method	What it does	Example on L = [3, 1, 2]
		5]
L.insert(i, x)	Inserts x at position i	L.insert(0, 9) → [9, 3, 1, 2]
L.remove(x)	Removes the first occurrence of the value	L.remove(1) → [3, 2]
L.pop(i)	Removes and returns the item at i (last by default)	L.pop() → 2
L.index(x)	Position of the first occurrence	L.index(2) → 2
L.count(x)	How many times the value occurs	L.count(3) → 1
L.sort()	Sorts the list in place	L.sort() → [1, 2, 3]
L.reverse()	Reverses the list in place	L.reverse() → [2, 1, 3]
L.clear()	Removes everything	L.clear() → []

append versus extend: L.append([4, 5]) adds one item that happens to be a list, giving [3, 1, 2, [4, 5]]. L.extend([4, 5]) adds two items. Mixing these up is one of the most common bugs in this lab.

9.2 Sorting

sort() rearranges the list itself and returns nothing; sorted() leaves the original alone and hands back a new list. Both accept reverse=True.

```
marks = [78, 45, 90, 62]

print(sorted(marks))           # a new sorted list
print(sorted(marks, reverse=True))
print(marks)                   # the original is untouched

marks.sort()                   # sorts in place, returns None
print(marks)
```

Output

```
[45, 62, 78, 90]
[90, 78, 62, 45]
[78, 45, 90, 62]
[45, 62, 78, 90]
```

A classic mistake: marks = marks.sort() throws your data away, because sort() returns None. Either call marks.sort() on its own line, or assign the result of sorted(marks).

9.3 Copying versus Aliasing

Assigning a list to a second name does **not** copy it. Both names refer to the same object in memory, so a change made through one is visible through the other. Use a slice, list() or copy() when you need an independent copy.

```
original = [1, 2, 3]

alias = original               # the same list under a second name
copy = original[:]             # an independent copy (list(original) also works)

alias.append(99)
```

```
print("original:", original)
print("alias   :", alias)
print("copy    :", copy)
```

Output

```
original: [1, 2, 3, 99]
alias    : [1, 2, 3, 99]
copy     : [1, 2, 3]
```

9.4 Slice Assignment and Two-Dimensional Lists

```
items = [1, 2, 3, 4, 5]
items[1:3] = [20, 30]      # replace a whole slice
print(items)

matrix = [[1, 2, 3],
          [4, 5, 6],
          [7, 8, 9]]

print(matrix[1])           # the second row
print(matrix[1][2])        # row 1, column 2

for row in matrix:         # walking a matrix row by row
    print(sum(row), end=' ')
```

Output

```
[1, 20, 30, 4, 5]
[4, 5, 6]
6
6 15 24
```

10. Tuples

A tuple is an ordered sequence like a list, but it is **immutable**: once created, its items cannot be replaced, added or removed. That restriction is the whole point — it protects data that must not change, and it makes a tuple usable in places where a list is not allowed.

10.1 Creating Tuples

```
point    = (3, 4)
record   = ('22K-1234', 'Ali', 3.4)
single   = (5,)          # the comma makes it a tuple
notuple  = (5)            # without the comma this is just the number 5
empty    = ()

print(type(single), type(notuple))
print(len(record), record[1], record[-1])
```

Output

```
<class 'tuple'> <class 'int'>
3 Ali 3.4
```

10.2 Packing and Unpacking

Several values written with commas are **packed** into a tuple, and a tuple can be **unpacked** into several names in one statement. This is what makes the swap from Lab 01 work, and it is how a function returns more than one value.

```

record = ('22K-1234', 'Ali', 3.4)
roll, name, cgpa = record          # unpacking
print(roll, name, cgpa)

a, b = 10, 20
a, b = b, a                        # pack on the right, unpack on the left
print(a, b)

# unpacking also drives the two-variable for loop you have been using
marks = {'Physics': 78, 'Maths': 90}
for subject, mark in marks.items():
    print(subject, mark)

```

Output

```

22K-1234 Ali 3.4
20 10
Physics 78
Maths 90

```

10.3 Where a Tuple Is Required

A dictionary key must be immutable, so a tuple can be a key while a list cannot. Tuples are also the conventional way to store fixed records — a coordinate, an RGB colour, a row read from a file.

```

distances = {('Karachi', 'Lahore'): 1210,
              ('Karachi', 'Quetta'): 690}

print(distances[('Karachi', 'Lahore')])

# a list of tuples is the standard shape for tabular data
students = [('Ali', 78), ('Sara', 92), ('Bilal', 65)]
for name, mark in students:
    print(name, mark)

```

Output

```

1210
Ali 78
Sara 92
Bilal 65

```

Try it: `{[1, 2]: 'x'}` raises `TypeError: unhashable type: 'list'`. Run it once so that you recognise the message when it appears in your own code.

11. Dictionaries

A dictionary stores values against **keys** rather than positions. Keys must be unique and immutable; values may be anything. Since Python 3.7 the insertion order of the keys is preserved.

11.1 The Methods You Will Actually Use

Method	What it does
<code>D[key]</code>	Reads the value — raises <code>KeyError</code> if the key is absent
<code>D.get(key, default)</code>	Reads the value, returning <code>default</code> instead of raising when the key is absent
<code>D[key] = value</code>	Adds a new pair, or overwrites an existing key
<code>D.setdefault(key,</code>	Inserts the pair only if the key is not already present

Method	What it does
value)	
D.update(other)	Merges another dictionary in, overwriting on collisions
D.pop(key)	Removes the key and returns its value
D.keys() D.values() D.items()	Views of the keys, the values, and the key–value pairs
key in D	Tests for a key — never searches the values
len(D)	The number of pairs

11.2 get() and the Missing Key

```
marks = {'Physics': 78, 'Maths': 90}

print(marks.get('Chemistry'))          # None instead of an error
print(marks.get('Chemistry', 0))       # a sensible default
print('Chemistry' in marks)

# marks['Chemistry'] # would raise KeyError: 'Chemistry'
```

Output

```
None
0
False
```

11.3 Iterating a Dictionary

```
marks = {'Physics': 78, 'Chemistry': 45, 'Maths': 90}

for subject in marks:                  # the keys, by default
    print(subject, end=' ')
print()

for mark in marks.values():
    print(mark, end=' ')
print()

for subject, mark in marks.items():    # both, unpacked
    print(subject, "->", mark)

print("average:", sum(marks.values()) / len(marks))
print("best   :", max(marks, key=marks.get))
```

Output

```
Physics Chemistry Maths
78 45 90
Physics -> 78
Chemistry -> 45
Maths -> 90
average: 71.0
best   : Maths
```

11.4 The Frequency-Counting Pattern

Counting how often each value occurs is the single most useful thing a dictionary does. The pattern is: for every item, take the count so far (defaulting to zero) and store it back increased by one.

```
sentence = "the lab is the first lab of the course"
counts = {}

for word in sentence.split():
    counts[word] = counts.get(word, 0) + 1

print(counts)
print("most frequent:", max(counts, key=counts.get))
```

Output

```
{'the': 3, 'lab': 2, 'is': 1, 'first': 1, 'of': 1, 'course': 1}
most frequent: the
```

11.5 Nested Dictionaries

A value may itself be a dictionary or a list, which is how records with several fields are stored.

```
students = {
    '22K-1234': {'name': 'Ali', 'marks': [78, 45, 90]},
    '22K-5678': {'name': 'Sara', 'marks': [88, 92, 95]},
}

for roll, info in students.items():
    average = sum(info['marks']) / len(info['marks'])
    print(roll, info['name'], round(average, 2))
```

Output

```
22K-1234 Ali 71.0
22K-5678 Sara 91.67
```

11.6 Sorting a Dictionary by Value

```
marks = {'Physics': 78, 'Chemistry': 45, 'Maths': 90}

for subject in sorted(marks, key=marks.get, reverse=True):
    print(subject, marks[subject])
```

Output

```
Maths 90
Physics 78
Chemistry 45
```

12. Sets

A set is an **unordered** collection of **unique** items. It cannot be indexed and it silently discards duplicates, which makes it the right tool for two jobs: removing repetition, and comparing two collections.

12.1 Creating and Changing a Set

```
marks = [78, 45, 90, 45, 78]

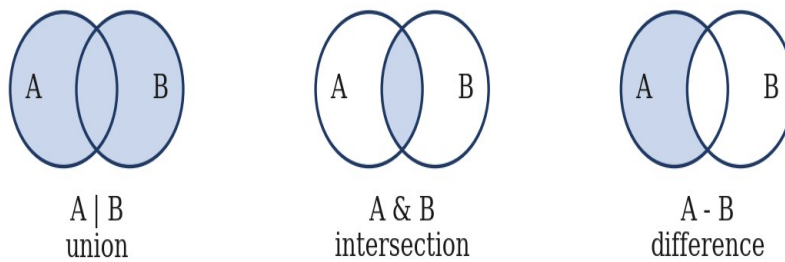
unique = set(marks)          # duplicates disappear
print(unique, len(unique))

empty = set()                # NOT {} – that is an empty dictionary
print(type(empty), type({}))
```

```
unique.add(60)           # add one item
unique.discard(45)       # remove without complaining if absent
print(unique)
print(78 in unique)
```

Output

```
{90, 45, 78} 3
<class 'set'> <class 'dict'>
{90, 60, 78}
True
```

12.2 Set Operations*Figure 12.1 Union, intersection and difference of two sets*

Operator	Method	Meaning
A B	A.union(B)	Everything in either set
A & B	A.intersection(B)	Only what appears in both
A - B	A.difference(B)	In A but not in B
A ^ B	A.symmetric_difference(B)	In one set or the other, but not both
A <= B	A.issubset(B)	Every item of A is also in B

```
section_a = {'Ali', 'Sara', 'Bilal', 'Hina'}
section_b = {'Sara', 'Hina', 'Usman'}

print("all students      :", section_a | section_b)
print("in both sections :", section_a & section_b)
print("only in A        :", section_a - section_b)
print("in exactly one   :", section_a ^ section_b)
```

Output

```
all students      : {'Ali', 'Sara', 'Hina', 'Bilal', 'Usman'}
in both sections  : {'Sara', 'Hina'}
only in A         : {'Ali', 'Bilal'}
in exactly one    : {'Ali', 'Bilal', 'Usman'}
```

12.3 What a Set Cannot Do

- It has no order, so `S[0]` is an error and the printed order may change between runs. Sort it into a list when order matters: `sorted(S)`.
- It cannot hold duplicates, so it is the wrong structure when repetition itself is the data — count with a dictionary instead.
- Its items must be immutable: numbers, strings and tuples are allowed, lists and dictionaries are not.

13. Choosing the Right Structure

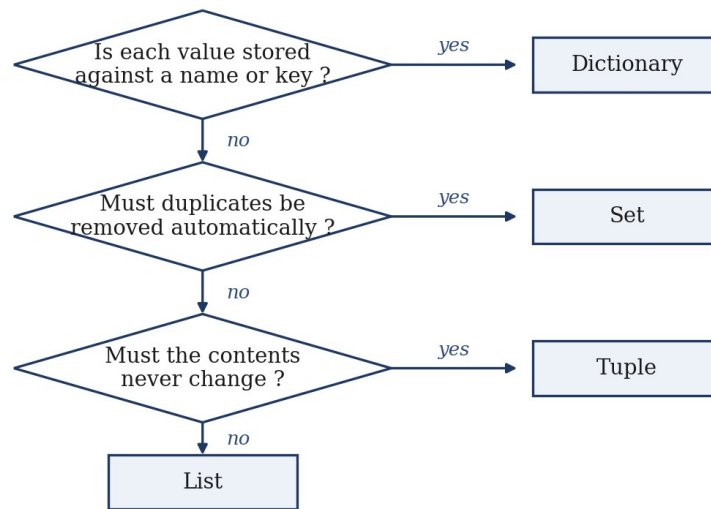


Figure 13.1 A quick decision chart for the four structures

	List	Tuple	Dictionary	Set
Written with	[]	()	{k: v}	{ } or set()
Ordered	Yes	Yes	By insertion	No
Changeable	Yes	No	Yes	Yes
Duplicates	Allowed	Allowed	Keys unique	Removed
Accessed by	Index	Index	Key	Membership only
Typical use	A growing sequence	A fixed record	A lookup table	Unique values

13.1 A Word on Performance

The structures differ not only in what they can hold but in how quickly they answer questions. The one that matters most in this course is membership: asking `x in L` on a list means comparing `x` against the items one after another, while asking `x in S` on a set or `k in D` on a dictionary jumps straight to the answer through a hash. On a handful of items the difference is invisible; on tens of thousands it is the difference between an instant answer and a noticeable wait.

Operation	List	Dictionary / Set	Comment
Access by index / key	Fast	Fast	Both jump straight to the item
<code>x in collection</code>	Grows with the size	Effectively constant	The reason to convert a large list to a set before repeated searching
Add one item at the end	Fast	Fast	<code>append()</code> for a list, <code>add()</code> or <code>D[k] = v</code> otherwise
Insert or delete in the middle	Slow — later items shift	Not applicable	Prefer a dictionary when items come and go by name

```

names = ['Ali', 'Sara', 'Bilal'] * 1000    # a long list with repeats
lookup = set(names)                       # build the set once

```

```
print('Sara' in names)      # scans the list
print('Sara' in lookup)    # one hash lookup – the shape to prefer
```

Rule of thumb: if a collection is searched repeatedly and its contents do not need an order, build a set from it once and search the set.

14. Quick Reference

Tool	Purpose	Example
<code>enumerate(seq)</code>	Index and item together	<code>for i, x in enumerate(L):</code>
<code>zip(a, b)</code>	Walk two sequences in step	<code>for k, v in zip(keys, vals):</code>
<code>break / continue</code>	Leave the loop / skip a pass	<code>if x < 0: continue</code>
<code>loop else</code>	Runs only if no break happened	<code>for ... : ... else: ...</code>
<code>[f(x) for x in L]</code>	List comprehension	<code>[n * 2 for n in L]</code>
<code>[x for x in L if c]</code>	Comprehension with a filter	<code>[n for n in L if n > 0]</code>
<code>{k: v for ... }</code>	Dictionary comprehension	<code>{w: len(w) for w in words}</code>
<code>{x for x in L}</code>	Set comprehension	<code>{n % 3 for n in L}</code>
<code>sorted(L, reverse=True)</code>	A new sorted list	<code>sorted(marks)</code>
<code>sorted(D, key=D.get)</code>	Keys ordered by their value	<code>max(D, key=D.get)</code>
<code>D.get(k, default)</code>	Safe lookup	<code>counts.get(w, 0) + 1</code>
<code>set(L)</code>	Remove duplicates	<code>len(set(L))</code>
<code>A B A & B A - B</code>	Union, intersection, difference	<code>set_a & set_b</code>
<code>L[:] list(L)</code>	An independent copy of a list	<code>backup = L[:]</code>

15. Class Tasks

These five tasks are to be completed **during the lab sessions** and demonstrated to the instructor before you leave — tasks C1 and C2 in the Lab 02 session, tasks C3 to C5 in the Lab 03 session. Work in the accompanying notebook `Lab02_03_Flow_and_Data_Structures.ipynb`, in the section titled **Class Tasks**.

C1. Number Analyzer — loop patterns, break and continue. Using a while loop, keep reading integers from the user until they enter 0, which acts as the sentinel. Ignore every negative number using `continue`, and leave the loop with `break` when the sentinel arrives. Store the accepted numbers in a list and then report: how many numbers were accepted, their sum and average, the largest and the smallest, how many were even, and how many were skipped for being negative. If no valid number was entered at all, print a message instead of dividing by zero.

Use each of the patterns from Section 7 exactly once, and do not use the built-in `max()` or `min()` — track them yourself in the loop.

C2. Comprehension Workshop — rewriting loops as one-liners. Start from the two lists given below. First write each of the five results with an ordinary loop, then rewrite each one as a single comprehension, and print both to prove they agree.

```
numbers = [12, 7, 40, 3, 25, 18, 7, 40] and words = ['python', 'ai', 'lab',
'data', 'comprehension']
```

(i) the squares of the odd numbers; (ii) the words longer than three letters, in capitals; (iii) a dictionary mapping each word to its length; (iv) the set of unique first letters of the words; (v) a list labelling each number "even" or "odd" using a conditional expression.

Finish with a one-line comment naming the one result above that you think is **clearer** as an ordinary loop, and why.

C3. Student Records — lists of dictionaries. Build a list of at least five student records, each a dictionary with the keys `roll`, `name` and `marks` (a list of three subject marks). Then, using loops and comprehensions, print: every student with their total and average; the name of the topper; the names of all students whose average is below 50; and a new list of (`name`, `average`) tuples sorted from the highest average to the lowest.

Hint: build the list of tuples first, then sort it — a list of tuples sorts on the first item, so store the average first if you want to sort on it.

C4. Two Sections — set operations. Two lab sections have submitted their attendance as lists of names, and both lists contain accidental duplicates. Convert them to sets and report: the complete list of distinct students; the students who attended both sections; those who attended only the first; those who attended exactly one of the two; and whether every name in the second section also appears in the first. Finally, print how many duplicate entries the original lists contained altogether.

```
section_a = ['Ali', 'Sara', 'Bilal', 'Ali', 'Hina'] and section_b = ['Sara',
'Hina', 'Usman', 'Sara']
```

C5. Word Frequency Report — dictionaries end to end. Take a sentence from the user and produce a small report: the number of words and the number of **distinct** words; a dictionary of word to frequency built with the counting pattern of Section 11.4; the words that occur more than once; the longest word; and a dictionary comprehension mapping each distinct word to its length. Print the frequency table sorted from the most frequent word to the least, and treat the input case-insensitively so that `The` and `the` count as the same word.

Assessment of class tasks: you must be able to explain any line of your code to the instructor, including why you chose a list, a dictionary or a set in each place. A working program that you cannot explain earns no marks.

16. Home Tasks

Attempt all the questions below. Write each program in a **separate .py file**, test it with more than one input, and submit it according to the naming convention in Section 17.

- Q1.** Take a list of numbers from the user and print a new list containing only the numbers that are divisible by 3 — once using an ordinary loop, and once using a list comprehension.
- Q2.** Write FizzBuzz for the numbers 1 to 50 as a **single list comprehension** that produces a list of strings, using a nested conditional expression, and then print the list one item per line.
- Q3.** Take a list from the user and remove all the duplicate values from it **while preserving the original order**. Print the result together with the number of duplicates that were removed. (A set alone will not preserve the order — think about how to use one as a record of what you have already seen.)
- Q4.** Take a dictionary of subject names and marks, and print an inverted dictionary in which the marks are the keys and the subject names are the values. State in a comment what happens when two subjects share the same marks.
- Q5.** Print all the prime numbers between 1 and 100 — first with nested loops and a `break`, and then, in the same file, as a single list comprehension. Compare the two versions in a comment.
- Q6.** Using nested loops, print the following two patterns for a height entered by the user: a left-aligned triangle of stars, and a centred pyramid.

- Q7.** Take two dictionaries of subject marks and merge them into one. When a subject appears in both, the merged dictionary must hold the **sum** of the two marks, not just the later one.
- Q8.** A list of (name, marks) tuples is given. Convert it into a dictionary, then print the names of all the students who scored above the class average, in descending order of marks.
- Q9.** Take three lists of numbers from the user and, using set operations, print: the values common to all three; the values that appear in exactly one of them; and the complete set of distinct values.
- Q10.** Take a sentence from the user and, using a dictionary comprehension, print how many vowels and how many consonants each word contains.
- Q11.** Write a menu-driven shopping cart. The cart is a dictionary of item name to quantity, and the menu — add an item, remove an item, view the cart, view the total number of items, exit — repeats inside a `while` loop until the user chooses to exit. Adding an item that is already in the cart must increase its quantity rather than replace it.
- Q12.** A matrix is given as a list of lists. Using a nested comprehension, print its transpose (rows become columns). Then print the sum of each row and the sum of each column.

17. Submission and Assessment

Follow the naming convention below so that your work can be identified and graded correctly:

- **Class tasks:** completed in the notebook `RollNo_Lab02_03_ClassTasks.ipynb` and demonstrated before the end of the session. Run **Kernel ► Restart & Run All** before you call the instructor.
- **Home tasks:** one script per question, named `RollNo_Lab02_03_Qn.py` — for example `22K-1234_Lab02_03_Q5.py`.
- Place all the files for this lab in a single folder and compress it before uploading.
- Every file must begin with a comment giving your name, roll number and the question number.
- Code that does not run will not be marked. Test your program before calling the instructor.

Assessment Criterion	Weight
Correctness of output for all valid inputs	35%
Class tasks completed and demonstrated in the session	20%
Correct choice of data structure and control flow for the problem	20%
Idiomatic use of comprehensions where they improve clarity	15%
Readability: meaningful variable names, indentation, comments	10%

End of Lab 02 & 03